RICE UNIVERSITY

Networking: Latency, Bandwidth, and Sockets

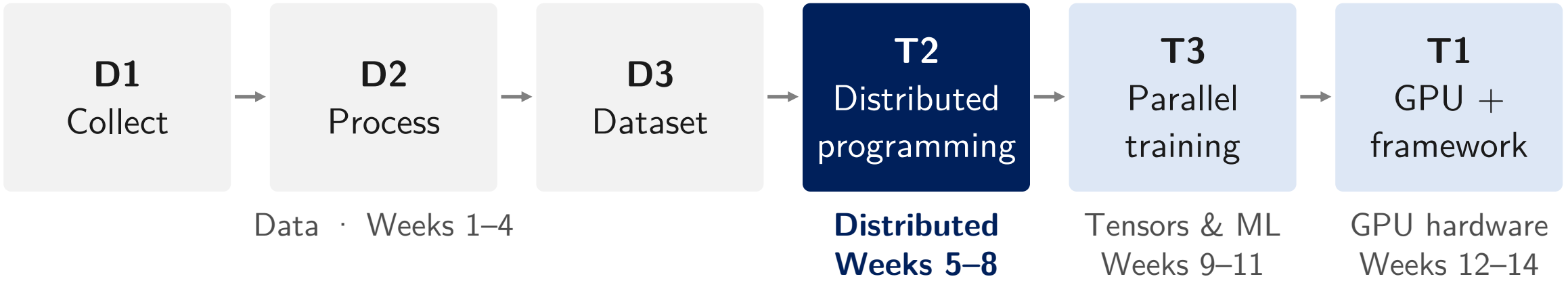
What a message costs, and how two programs on different machines talk

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 6, Lecture 1

Where we are in the pipeline



In our pipeline

Today: what it costs to move bytes **between machines**. D3 checkpoint this week; T2 builds on today's $\alpha + \beta n$ model.

Last time (Week 5): processes and threads on one machine. Next: serialization and RPC (Week 6 L2).

Why do we need networks?

The job: servers training a 7.5-billion-parameter model
exchange **15 GB** of gradients (BF16) after **every** step

How long does one copy take over ordinary 1 Gb/s Ethernet?

1 Gb/s Ethernet (125 MB/s)
$\approx$ **120 s** per step

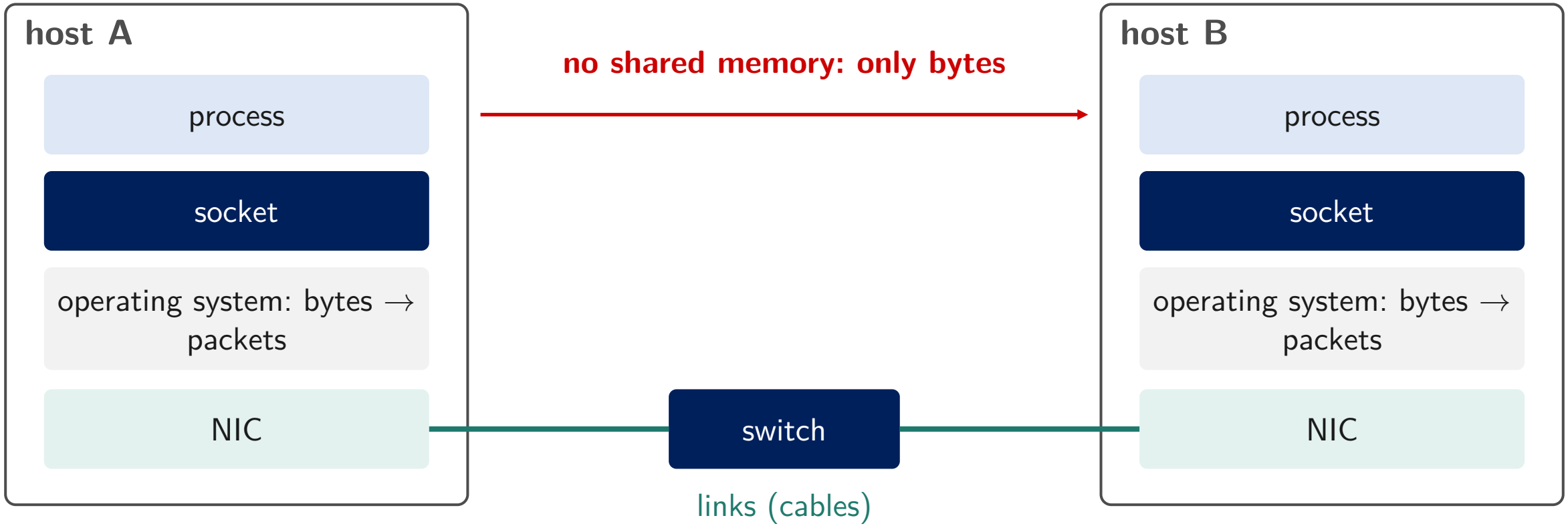
400 Gb/s InfiniBand (50 GB/s)
$\approx$ **0.3 s** per step

Key idea

Once work spans machines, every byte crosses a network, and its cost decides whether **more machines help**.

Source: IEEE 802.3ab (1 Gb/s); InfiniBand Trade Association (NDR 400 Gb/s); ZeRO, SC 2020; times derived.

What a network does, in plain words



Key idea

A **network** copies bytes between processes on different hosts. **Nothing else** is shared.

Where networks show up

- **Spark (Weeks 2–4):** the shuffle sends rows between workers.
- **RPC (Week 6 L2):** call a function on another machine.
- **mpi4py, NCCL (Weeks 7–8):** messages and collectives for training; T2.
- **Everywhere else:** the web, SSH, databases — all over sockets.

The price

Every message costs **start-up time** + **time per byte**

Messages can be **lost** or **reordered**

Machines **fail** independently

Why we need it

Why networks? The work no longer fits on one machine. Today: what a message costs, and the socket interface every later tool sits on.

Source: COMP 329 syllabus, Spring 2027; Zaharia et al., NSDI 2012 (Spark); NVIDIA NCCL User Guide.

Learning goals for today

- **Distinguish** latency from bandwidth, and say where each comes from.
- **Model** a message as $\alpha + \beta\mathbf{n}$, and find the size $n_{1/2}$ where latency stops dominating.
- **Use** sockets to send bytes, and **choose** between TCP and UDP.
- **Predict** a ping-pong time before measuring it.

A puzzle to start

Link A: fast but far
first byte after 50 ms, then 1 Gb/s

Link B: slow but near
first byte after 0.5 ms, then 100 Mb/s

Which link delivers a 10 KB web page sooner? And a 1 GB file?

Answer

10 KB: **B** (1.3 ms vs 50 ms). 1 GB: **A** (8 s vs 80 s). Which link is faster **depends on the size**.

Today's plan

1

Latency and bandwidth

Two numbers describe any link. Where does each come from?

2

The $\alpha + \beta n$ model

Two numbers predict the time of any message.

3

Sockets, TCP, and UDP

How does a program actually send bytes to another machine?

Build

TCP ping-pong, UDP burst

Pairs: predict with $\alpha + \beta n$ first, then measure.

Part 1

Latency and bandwidth

Two numbers describe any link. Where does each come from?

1 · Latency and bandwidth

2 · The $\alpha + \beta n$ model

3 · Sockets, TCP, and UDP

Build · TCP ping-pong, UDP burst

Latency versus bandwidth

narrow, short pipe



wide, long pipe



latency (seconds): time for the first byte to arrive

bandwidth (bytes/s): how much flows per second once it runs

Key idea

Latency and bandwidth are **independent**. Network speeds use **bits**/s: divide by 8.

Where latency comes from

Distance	light in fibre: $\approx$ **5 µs per km**
Switches	each hop stores, forwards, and may **queue** the packet
Software	OS calls and copies: $\approx$ **24 µs** on our laptop, with **no wire**

Houston $\rightarrow$ New York $\geq$ **11 ms** · across a datacenter $\approx$ **2.4 µs**

Key idea

Over long distances, **physics** sets latency. Inside a datacenter, **software** does.

Source: speed of light; Corning SMF-28 datasheet (group index 1.468); distances computed; software: our laptop.

Bandwidth: line rate and headers

One Ethernet frame on the wire (bytes):

preamble 8	Ethernet 14	IP 20	TCP 20	your data: 1,460	check 4	gap 12

$$\text{efficiency} = 1{,}460 \;/\; 1{,}538 \approx 94.9\%$$

1 Gb/s line rate = 125 MB/s $\rightarrow$ at most $\approx$ **119 MB/s** of your data

Key idea

The line rate is an upper bound; **headers** and **software** lower what you measure.

Source: IEEE 802.3 (line rate, frame, preamble, gap); RFC 791 (IPv4 header); RFC 9293 (TCP header).

Check yourself

Two minutes with a neighbour.

- A network card is sold as **400 Gb/s**. How many GB/s is that?
- Two datacenters are 1,000 km apart. What latency can no hardware remove?
- Which helps a 10 KB request more: double the bandwidth or halve the latency?

Part 2

The $\alpha + \beta n$ model

Two numbers predict the time of any message.

1 · Latency and bandwidth

2 · The $\alpha + \beta n$ model

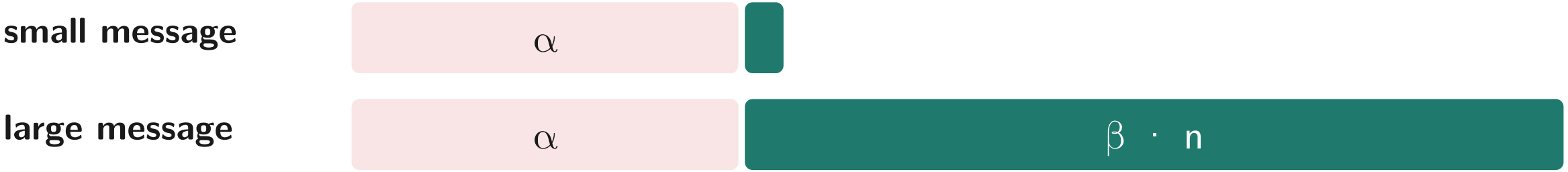
3 · Sockets, TCP, and UDP

Build · TCP ping-pong, UDP burst

The cost of one message

$$T(n) = \alpha + \beta \cdot n$$

- α (latency): start-up cost of **every** message, in seconds.
- β (time per byte), in seconds per byte: β **= 1 / B**, with B the bandwidth.



Key idea

Every message pays α **once**, then β **per byte**. (From Week 7 on: $\alpha + n/\beta$.)

Source: Hockney, "The communication challenge for MPP," Parallel Computing 20(3), 1994; Thakur, Rabenseifner & Gropp, IJHPCA 2005.

A worked example

$$\alpha = 25\ \mu\text{s}, \quad \beta = 1\ /\ (125\ \text{MB/s}) = 8\ \text{ns per byte}$$

$$n = 100\ \text{bytes}: \quad T = 25\ \mu\text{s} + 0.8\ \mu\text{s} \approx 26\ \mu\text{s}$$

$$n = 1\ \text{MB}: \quad T = 25\ \mu\text{s} + 8\ \text{ms} \approx 8\ \text{ms}$$

Key idea

100 bytes: α is 97% of the time, **latency-bound**.

1 MB: β**n** is 99.7%, **bandwidth-bound**.

Source: α assumed (Kalia et al., NSDI 2019: 10–100 µs commonly assumed for kernel TCP); β: IEEE 802.3ab.

Where small turns into large: $n_{1/2}$

$$\alpha = \beta \cdot n_{1/2} \quad \Leftrightarrow \quad n_{1/2} = \alpha \,/\, \beta = \alpha \cdot B$$

$$\text{at } n = n_{1/2}\text{:} \quad n \,/\, T(n) = n_{1/2} \,/\, (2\alpha) = B \,/\, 2$$

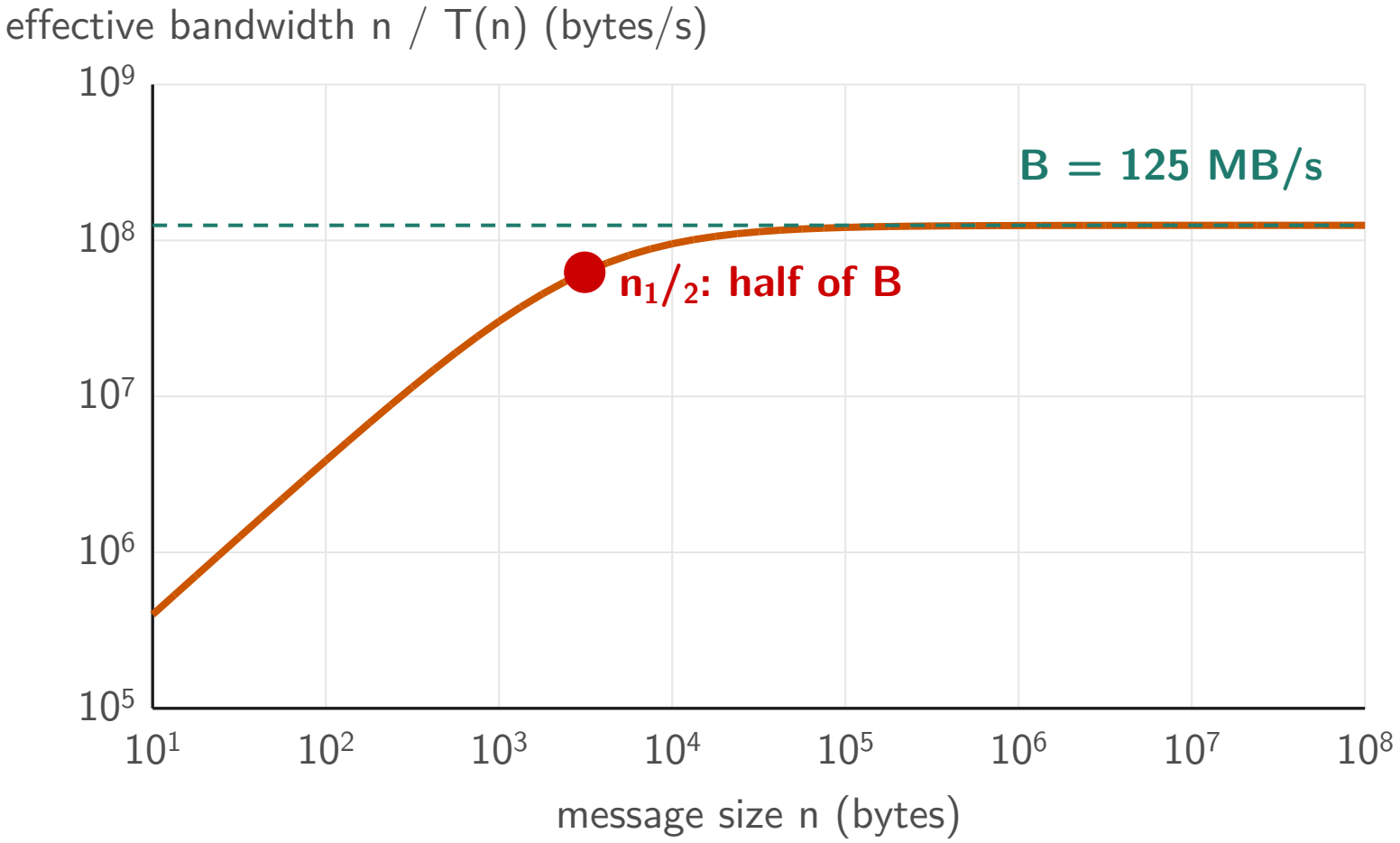
$$\alpha = 25\ \mu\text{s},\ \ B = 125\ \text{MB/s} \quad \Rightarrow \quad n_{1/2} \approx \mathbf{3.1\ KB}$$

Key idea

At $\mathbf{n_{1/2}}$ half the time is start-up and the link runs at **half its bandwidth**.

Source: Hockney, Parallel Computing 20(3), 1994 (half-performance message size).

Effective bandwidth



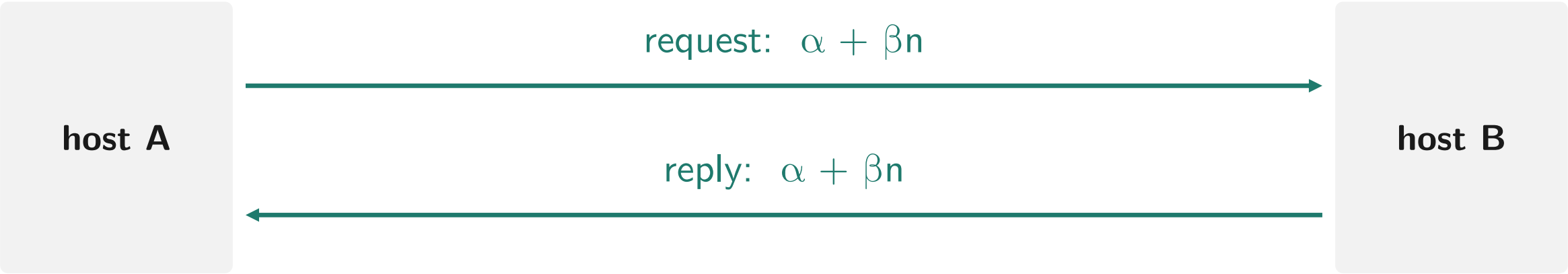
Two regimes

$n \ll n_1/_2$: $\approx n/\alpha$, **latency-bound**

$n \gg n_1/_2$: $\approx B$, **bandwidth-bound**

Source: Model curve: $\alpha = 25$ µs (assumed), B = 125 MB/s (IEEE 802.3ab); Hockney, Parallel Computing, 1994.

A round trip pays twice



$$\text{echo of } n \text{ bytes:} \quad \text{round trip} = 2\,(\alpha + \beta\ n)$$

Check

Time a **round trip on one clock**, then halve it: that is how we measure $\alpha + \beta n$.

Source: Hockney, "The communication challenge for MPP," Parallel Computing 20(3), 1994; Thakur, Rabenseifner & Gropp, IJHPCA 2005.

Check yourself

Two minutes with a neighbour.

- $\alpha = 25$ µs, B $= 100$ Gb/s $= 12.5$ GB/s. What are β and $n_1/_2$?
- Same link: 1,000 messages of 1 KB, or one of 1 MB. Estimate both.
- What happens to $n_1/_2$ if α halves? If the bandwidth doubles?

Part 3

Sockets, TCP, and UDP

How does a program actually send bytes to another machine?

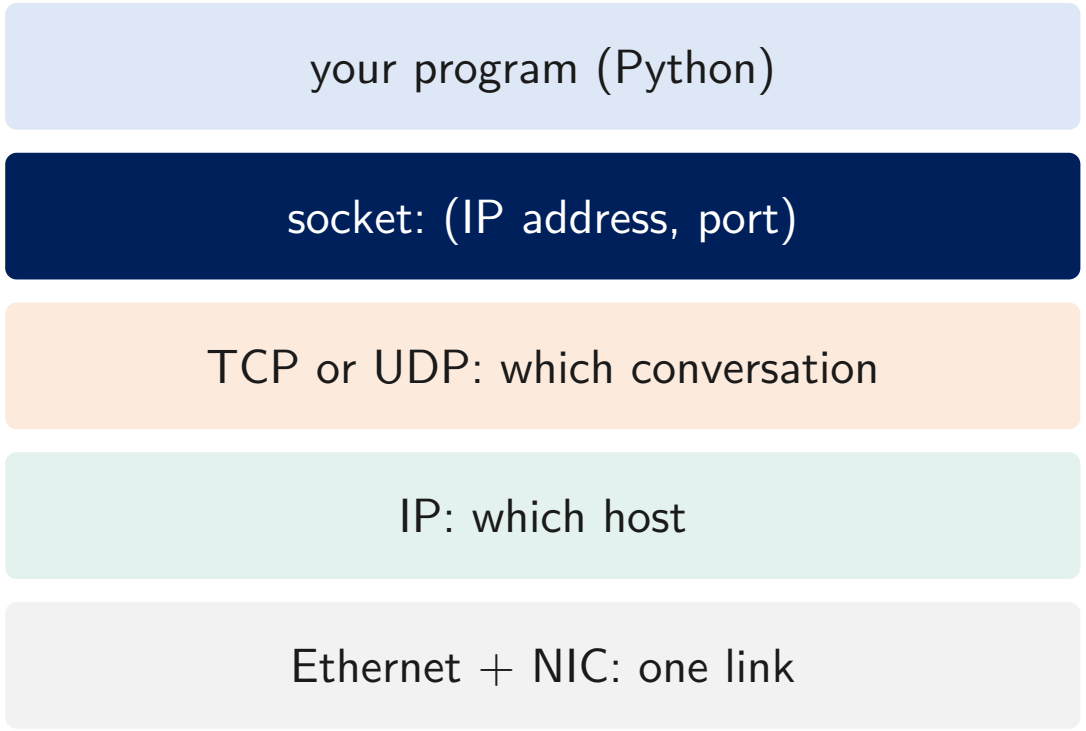
1 · Latency and bandwidth

2 · The $\alpha + \beta n$ model

3 · Sockets, TCP, and UDP

Build · TCP ping-pong, UDP burst

A socket is one end of a conversation



address: 127.0.0.1 = this machine (loopback)

port: which program, 0–65535

each layer adds its header on the way down

Key idea

A **socket** is a file-like OS object named by (**IP address, port**). Programs see only bytes.

Source: Python 3 documentation, socket module; RFC 791 (IP); RFC 9293 (TCP); RFC 768 (UDP).

Five calls are enough

```
# server (host B)
srv = socket.create_server(("", 5000))
conn, addr = srv.accept()   # wait for a client
data = conn.recv(1024)      # at most 1024 bytes
conn.sendall(data)          # echo it back
```

```
# client (host A)
s = socket.create_connection((HOST, 5000))
s.sendall(b"hello")
print(s.recv(1024))         # b'hello'
```

- **create_server**: listen on a port
- **accept**: one socket per client
- **create_connection**: dial
- **sendall / recv**: move bytes
- **close**: hang up

Source: Python 3 documentation: socket — Low-level networking interface; Socket Programming HOWTO.

TCP is a byte stream, not messages

```
def recv_exact(s, n):
    buf = bytearray()
    while len(buf) < n:
        part = s.recv(n - len(buf))
        if not part:                    # peer closed
            raise ConnectionError
        buf += part
    return bytes(buf)
```

sent:	1,000 B	1,000 B
recv:	1,460 B	540

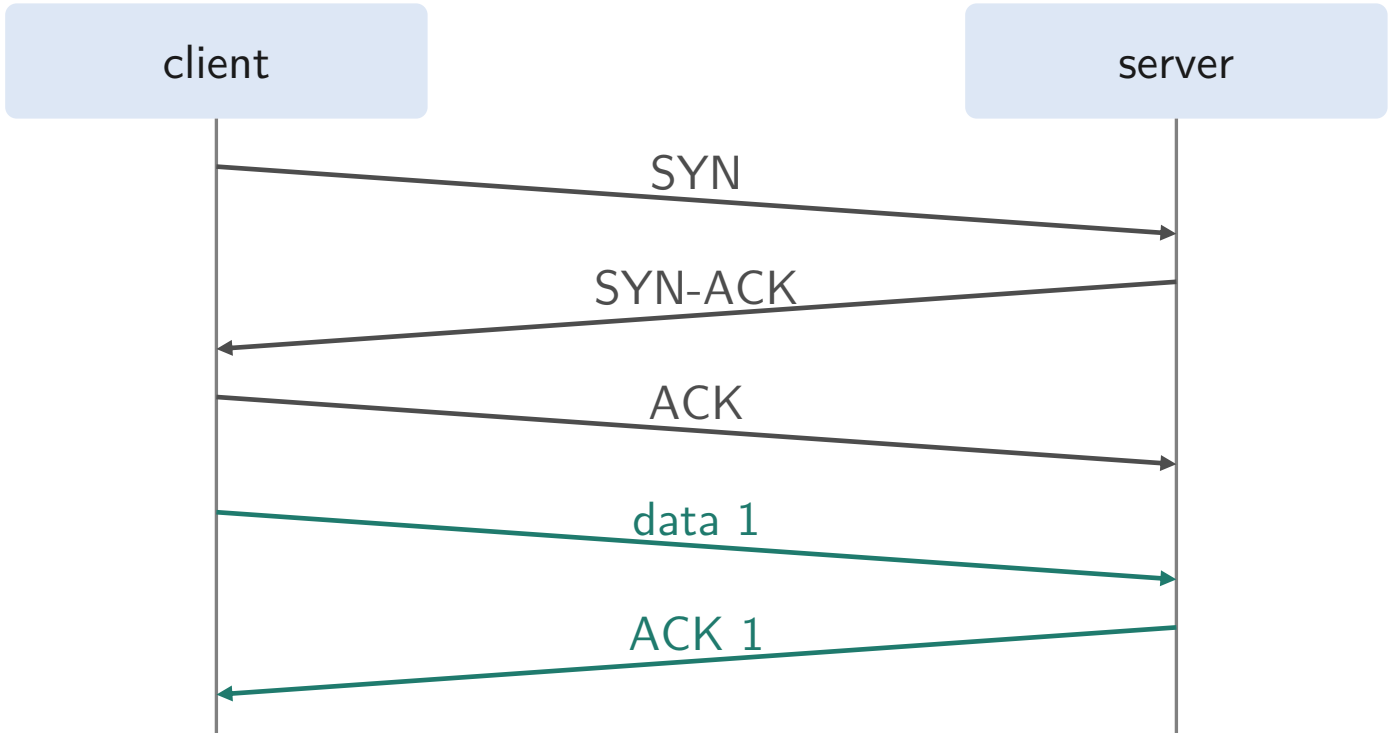
same bytes, different pieces

Watch out

`recv(n)` may return **fewer** than n bytes. Loop until you have them all.

Source: Python Socket Programming HOWTO; RFC 9293.

TCP: a reliable, ordered connection



- **Connection**: a handshake first, one extra round trip.
- **Order**: sequence numbers; bytes arrive in order.
- **Reliability**: ACKs; lost data is **resent**.

Key idea

TCP hides loss by **resending**. The price: extra round trips and waiting.

Source: RFC 9293, Transmission Control Protocol (2022).

Head-of-line blocking



Watch out

One lost packet **stalls everything behind it** until it is resent: **head-of-line blocking**.

Source: RFC 9293 (TCP); RFC 9000 (QUIC), on head-of-line blocking.

UDP: send a datagram and hope

```
u = socket.socket(type=socket.SOCK_DGRAM)
u.sendto(b"hello", (HOST, 5001))   # no connect

r = socket.socket(type=socket.SOCK_DGRAM)
r.bind(("", 5001))
data, sender = r.recvfrom(65535)  # one datagram
```

- **One datagram** per send; boundaries kept.
- **No** connection, ACK, resend, or order.
- At most **65,507 bytes** (IPv4).

Watch out

UDP is **cheap and unreliable**: the network may drop or reorder datagrams, and nobody tells you.

Source: RFC 768 (UDP); RFC 791 (IPv4); Python 3 documentation: socket — Low-level networking interface; Socket Programming HOWTO.

TCP or UDP?

	TCP	UDP
Connection	yes: handshake first	no
Delivery	reliable, in order	may drop or reorder
Boundaries	a byte stream	one datagram per send
Used by	web, SSH, MPI, Spark	DNS, video calls, QUIC

Key idea

TCP when every byte matters (everything in this course). **UDP** when late data is worthless, or you add reliability yourself.

Source: RFC 9293 (TCP); RFC 768 (UDP); RFC 9000 (QUIC).

Check yourself

Two minutes with a neighbour.

- A client calls `sendall` with 4,000 bytes. Does one `recv(4096)` on the server get them all?
- Sending a 1 MB array of gradients: TCP or UDP? Why?
- Why does opening a **new** TCP connection per message add latency?

IN-CLASS BUILD

TCP ping-pong, UDP burst

Pairs: predict with $\alpha + \beta n$ first, then measure.

1 · Latency and bandwidth

2 · The $\alpha + \beta n$ model

3 · Sockets, TCP, and UDP

Build · TCP ping-pong, UDP burst

Exercise 1: a TCP echo server

server.py

```
srv = socket.create_server(("", PORT))
conn, addr = srv.accept()
conn.setsockopt(socket.IPPROTO_TCP,
                socket.TCP_NODELAY, 1)
for n in SIZES:
    for _ in range(REPS):
        conn.sendall(recv_exact(conn, n))
```

DO

1. Read the code: what does the server do for each size?
2. Start it in terminal 1.

ASK

Why does the server read the **whole** message before echoing it?

Source: Python 3 documentation: socket — Low-level networking interface; Socket Programming HOWTO.

Exercise 2: the ping-pong client

client.py

```
s = socket.create_connection((HOST, PORT))
s.setsockopt(socket.IPPROTO_TCP,
             socket.TCP_NODELAY, 1)
for n in SIZES:
    msg = bytes(n)
    t0 = time.perf_counter()
    for _ in range(REPS):
        s.sendall(msg); recv_exact(s, n)
    t = (time.perf_counter() - t0) / REPS / 2
    print(n, t)                          # one-way
```

DO

1. Measure 1 B and 1 MiB: get α and β.
2. **Predict** 64 KiB and 4 MiB.
3. Run all sizes. How close?

ASK

Compute $n_{1/2} = \alpha/\beta$. Which sizes are latency-bound?

Source: Python 3 documentation: socket — Low-level networking interface; Socket Programming HOWTO.

Exercise 3: a burst over UDP

```
# receiver.py (start first)
r = socket.socket(type=socket.SOCK_DGRAM)
r.bind(("", 5001)); r.settimeout(1.0)
got = 0
try:
    while True:
        r.recv(65535); got += 1
except TimeoutError:
    print("received", got)
# sender.py
u = socket.socket(type=socket.SOCK_DGRAM)
msg, t0 = bytes(1024), time.perf_counter()
for i in range(10_000):
    u.sendto(msg, (HOST, 5001))
print((time.perf_counter() - t0) / 10_000)
```

DO

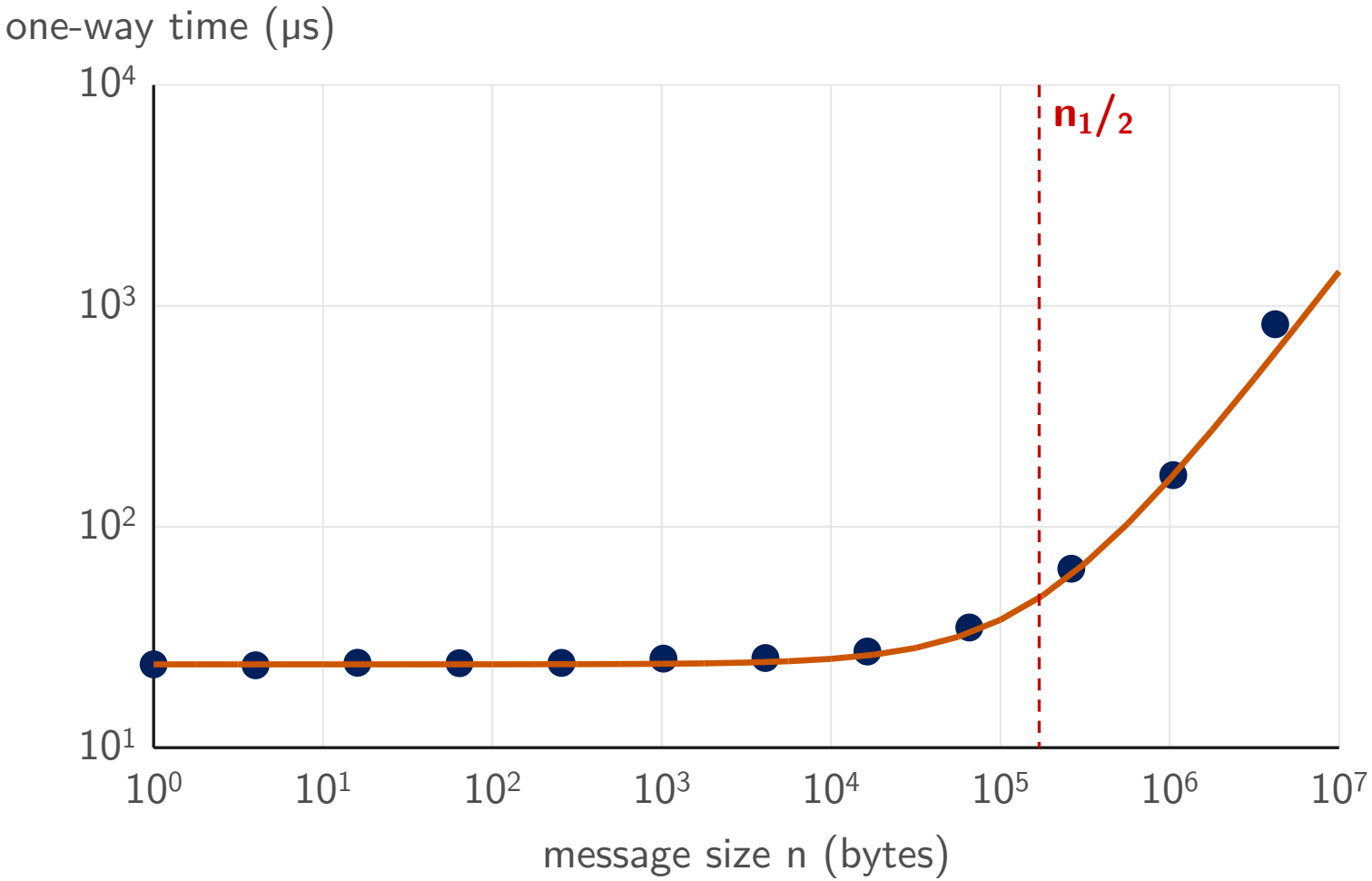
1. **Predict** the time per send and how many of 10,000 arrive.
2. Run receiver, then sender.
3. Run it three times. Same count?

STRETCH

Number the datagrams. Do they arrive in order?

Source: Python 3 documentation: socket — Low-level networking interface; Socket Programming HOWTO. RFC 768.

Solutions: the ping-pong



Our laptop

$\alpha \approx$ **24 µs**

B $\approx$ **7.1 GB/s**

$n_{1/2} \approx$ **169 KB**

dots: measured

orange: $\alpha + \beta n$

Source: one laptop run made for these slides: Apple M1 Pro, macOS, Python 3.14, loopback (127.0.0.1); median of 5 runs.

Solutions: predict, then measure

$$\text{64 KiB:} \quad \text{predicted } 33\ \mu\text{s}, \quad \text{measured } 35\ \mu\text{s}$$

$$\text{4 MiB:} \quad \text{predicted } 614\ \mu\text{s}, \quad \text{measured } 826\ \mu\text{s}$$

Check

Two numbers predict 64 KiB within **6%**; at 4 MiB the copies leave the caches and we are **26%** low. Week 7 fits α and β properly.

Source: one laptop run made for these slides: Apple M1 Pro, macOS, Python 3.14, loopback (127.0.0.1); median of 5 runs.

Solutions: the UDP burst

- Time per send $\approx$ **28 µs**: about one α, and **no reply** to wait for.
- Received **9,489–9,756** of 10,000; **no error** anywhere.
- In order on loopback, but nothing **promised** it.

Watch out

UDP is cheap until you need every byte. Then you rebuild TCP: **number, acknowledge, resend**.

Source: one laptop run made for these slides: Apple M1 Pro, macOS, Python 3.14, loopback (127.0.0.1); median of 5 runs.

Key takeaways

- **Latency** (time to first byte) and **bandwidth** (bytes per second) are independent.
- Over distance physics sets latency; in a datacenter **software** does.
- $\mathbf{T(n) = \alpha + \beta n}$, $\beta = 1/\mathrm{B}$; below $\mathbf{n_1/_2 = \alpha/\beta}$ a message is latency-bound.
- A **socket** is (address, port); TCP is a reliable, ordered **byte stream**.
- **UDP** keeps message boundaries but may drop or reorder; nobody tells you.

Next time, and readings

When	What
Week 6, Lecture 2	Serialization, RPC, and datacenter networks
This week	D3 checkpoint
Week 7, Lecture 1	Message passing with mpi4py; fit α and β

READINGS

Python documentation: "Socket Programming HOWTO".

Hockney, "The communication challenge for MPP," Parallel Computing, 1994 (the $\alpha + \beta n$ model).